

Reflektionsrapport

[Users & Profiles REST API](#)

Systematisk programutveckling med Java | CAW24G

Michiel van der Gragt

Vad har jag byggd för applikation?

Jag byggde ett REST API som skapar, läsar, uppdaterar och tar bort användare och deras profiler. API:et byggdes med Java och Spring Boot. Du kan testa det med ett verktyg som Insomnia genom att skicka förfrågningar till adresser som: *GET /api/users/1*

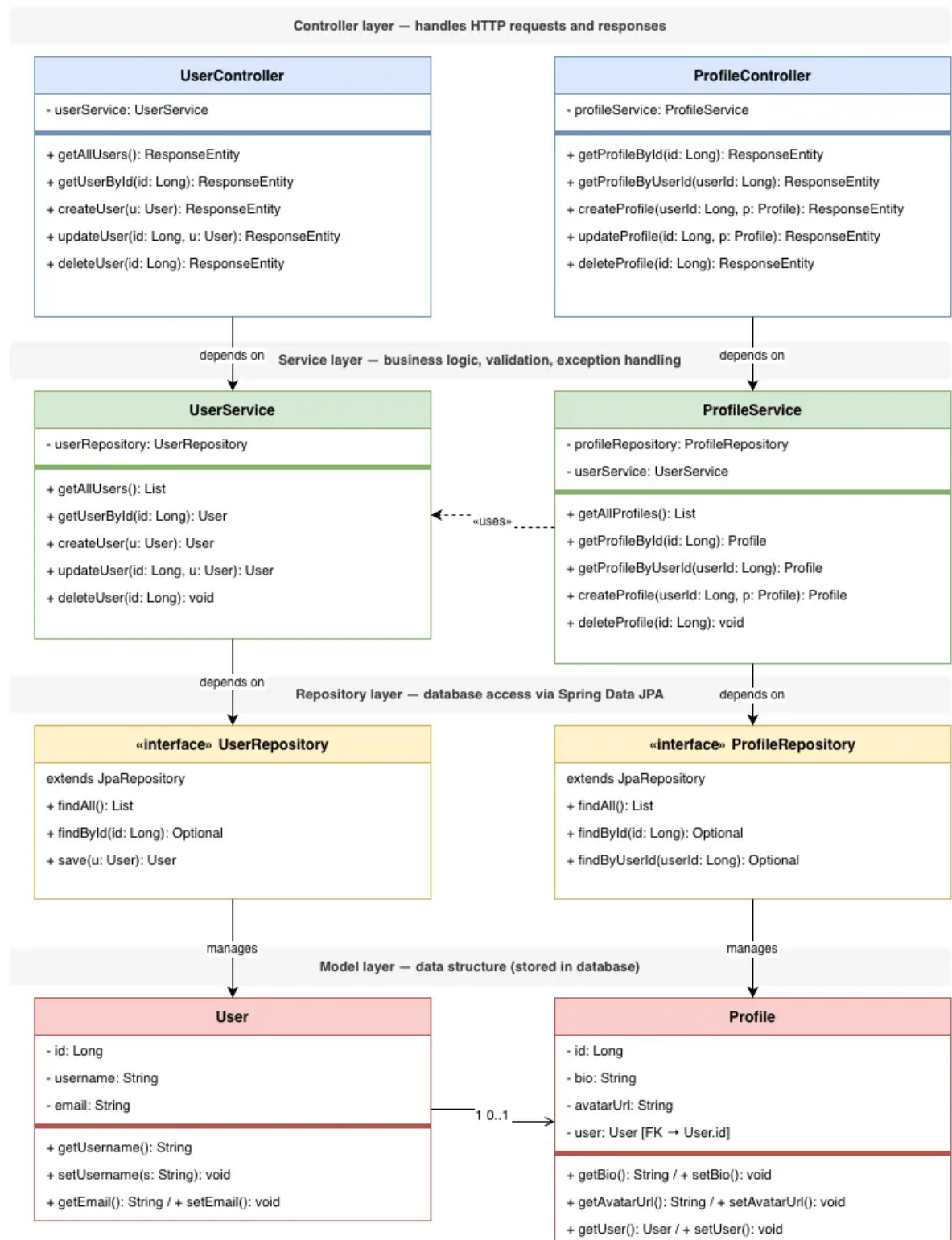
Hur är koden strukturerad?

Koden är uppdelad i fyra lager. Varje lager har ett jobb och pratar bara med lagret direkt under sig.

Controller	Controllern är ytterdörren till appen. Den tar emot förfrågningar från omvärlden och skickar tillbaka svar. Den innehåller ingen logik, den skickar bara saker vidare.
Service	Servicen är hjärnan. Alla regler och all logik finns här. Till exempel, om du försöker hämta en användare som inte finns, kastar servicen ett felmeddelande.
Repository	Repositoryn är förrådet. Den pratar med databasen för att hitta, spara, uppdatera eller ta bort poster. Den innehåller ingen logik alls.
Model	Modellen är ritningen. Den beskriver hur en <i>User</i> och en <i>Profile</i> ser ut, vilka fält de har, som <i>username</i> , <i>email</i> och <i>bio</i> .

Jag valde den här strukturen för att varje lager är lätt att testa på egen hand. Om något går sönder vet du exakt vilket lager du ska titta i.

UML-modell med klassdiagram



Varför använde jag “constructor injection”?

Istället för att använda `@Autowired` på fält skickade jag alltid beroenden genom konstruktorn:

```
public UserService(UserRepository userRepository) {this.userRepository =  
userRepository;}
```

Jag gjorde detta för att det gör testning enklare. I tester kan du bara skicka in ett fejkat (mock) objekt direkt i konstruktorn. Det gör det också tydligt vad en klass behöver för att göra sitt jobb och du ser det direkt högst upp i klassen.

Varför använde jag final på fält?

Jag markerade alla injected fields som *final*:

```
private final UserRepository userRepository;
```

Det betyder att fältet bara kan sättas en gång, nämligen i konstruktorn. Det kan aldrig råkas ändras senare i koden.

Hur hänger User och Profile ihop?

En *User* kan ha noll eller en *Profile*. En *Profile* tillhör alltid exakt en *User*.

Jag höll dem som två separata klasser för att de lagrar olika saker. *User* lagrar inloggningsinformation (användarnamn och e-post). *Profile* lagrar extra information (bio och avatar). Om reglerna kring en av dem förändras behöver jag bara ändra en klass.

Varför använder ProfileService UserService?

När en profil skapas anropar *ProfileService* först *userService.getUserById()* för att kontrollera om användaren faktiskt finns. Om användaren inte finns kastas ett fel innan profilen försöker sparas.

Jag gjorde detta istället för att gå direkt till *UserRepository* för att logiken för att kontrollera om en användare finns ska finnas på ett ställe nämligen i *UserService*. Om jag någonsin ändrar den logiken behöver jag bara ändra den på ett ställe.

Hur hanteras fel?

Jag skapade en *GlobalExceptionHandler*-klass som fångar alla fel och returnerar ett tydligt JSON-svar:

```
{  
  "timestamp": "2026-04-06T12:00:00",  
  "status": 404,  
  "error": "Not Found",  
  "message": "User not found with id: 99"  
}
```

Utan detta skulle varje controller-metod behöva sin egen felhanteringskod. Genom att samla allt på ett ställe förblir koden ren och alla felsvar ser likadana ut.

Varför använde jag @Transactional?

Jag lade *@Transactional* på varje service-metod. Det betyder att om något går fel när data sparas ångras allt automatiskt. Ingenting sparas halvvägs.

För metoder som bara läser data (som *getAllUsers()*) använde jag *@Transactional(readOnly = true)*. Det berättar för databasen "jag läser bara, skriver inte", vilket gör de operationerna lite snabbare.

Varför använde jag Docker?

Docker paketerar appen så att den körs på samma sätt på alla datorer. Utan Docker kanske appen fungerar på min laptop men inte på någon annans eftersom de har en annan Java-version eller inställning.

Jag använde en tvåstegsbygge i Dockerfilen. Det första steget kompilerar koden med en fullständig Maven-bild. Det andra steget kör bara den färdiga appen med en liten, lätt bild. Det gör den slutliga Docker-bilden blir ungefär 75% mindre i storlek.

Varför använde jag GitHub Actions?

Jag satte upp en CI-pipeline som körs automatiskt varje gång jag pushar kod till GitHub. Den kör alla tester först och bygger bara Docker-bilden om alla tester godkänns.

Det betyder att jag aldrig råkar pusha trasig kod utan att veta om det. Pipelinen fungerar som en automatisk kvalitetskontroll på varje commit.

Vad lärde jag mig?

Det viktigaste jag lärde mig är att bra struktur gör allt enklare. Eftersom varje lager har ett jobb var testning enkel. Jag kunde testa servicen utan en igång databas och kontrollern utan riktig affärslogik.

Jag lärde mig också att små beslut som till exempel att använda constructor injection eller markera fält som *final*. Detta leder till kod som är mycket lättare att förstå och underhålla.